

Curso de Ciência de Dados, Machine Learning, Deep Learning e IA generativa com uso de LLMs

Disciplina: Visão Computacional e IA Generativa

Professor(a): João Macedo

Visão Computacional e IA Generativa

Apresentação

Visão Computacional e IA Generativa

Programa do Curso

- 1 - Fundamentos de Visão Computacional
- 2 - Aplicações de Deep Learning para Visão Computacional
- 3 - GANs e Diffusion Models
- 4 - Vision Transformers e Prompt-tuning
- 5 - Detecção e reconhecimento facial
- 6 - Deploy para edge (ONNX + TensorRT)

Visão Computacional e IA Generativa

1 - Fundamentos de Visão Computacional

1.1. O que é Visão Computacional?

1.2. Representação digital de imagens (PIL e OpenCV)

1.3. NumPy para manipulação de imagens

1.4. Histograma, contraste e realce

1.5. Transformações geométricas e filtros

1.6. Operadores morfológicos

Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?

Visão Computacional (Computer Vision / CV) é a área da computação que estuda como fazer computadores enxergarem e interpretar o mundo visual — imagens, vídeos, e representações 3D. Essa área atualmente é abrangida pelo campo da inteligência artificial.

Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?

Enquanto humanos interpretam imagens de forma instintiva e paralela, computadores precisam de **representações matemáticas** explícitas e **algoritmos** que **extraíam significado** a partir de números.

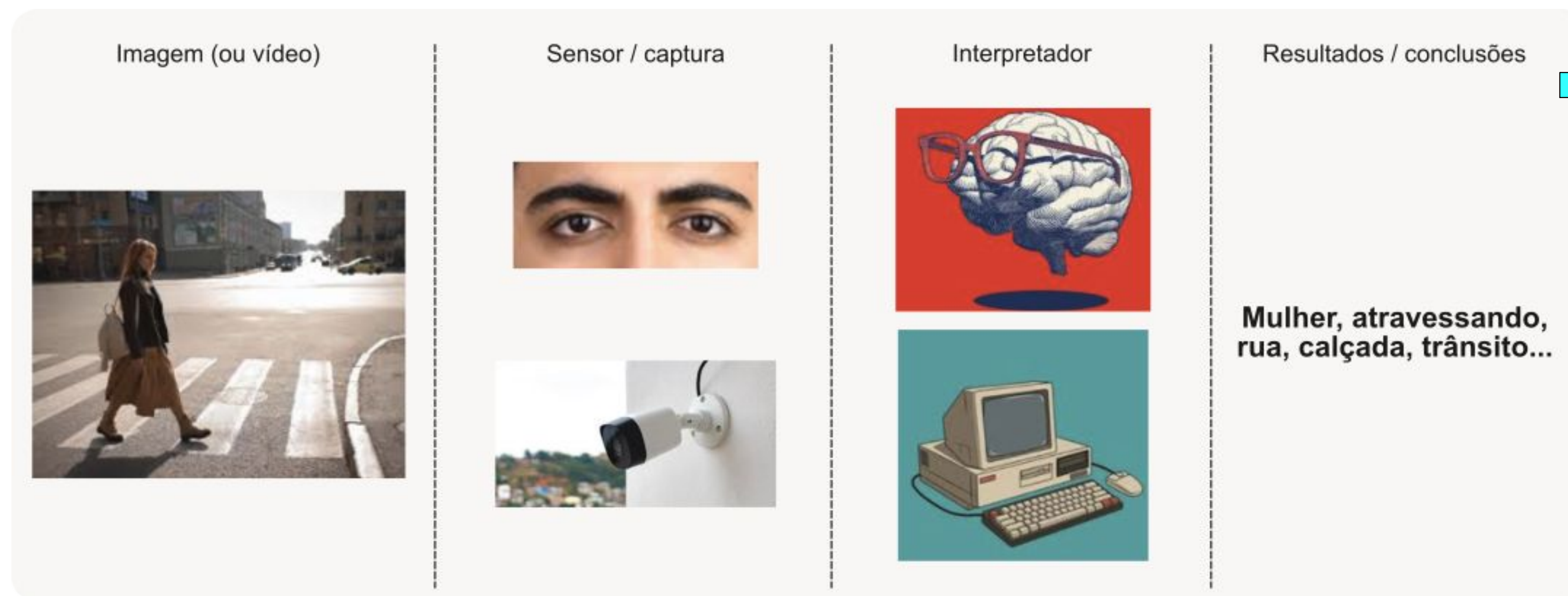
Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?

Diferente da **análise digital de imagens**, na **visão computacional** o objetivo é compreender o que ocorre na imagem, extrair informações, significado.

Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?



Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?

Aplicações gerais

Área	Exemplos de aplicação
Medicina	Diagnóstico por imagem (raios-X, ressonância, histopatologia)
Indústria	Controle de qualidade, detecção de defeitos
Segurança	Reconhecimento facial, detecção de atividade, monitoramento por câmeras
Veículos autônomos	Detecção de pedestres, sinais, faixas
Varejo	Checkout sem caixa, análise de gôndola
Agricultura	Detecção de pragas, estimativa de safra

Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?

Aplicações periciais

- Reconhecimento de pessoas em imagens e vídeos
- Classificação de imagens e vídeos ilegais (ex: imagens e vídeos de abuso sexual infantil)
- Análise de gemas
- Análise de imagens por satélite (dano ambiental, desmatamento, mineração ilegal, etc.)
- Análise de obras de arte
- Deepfakes

Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?

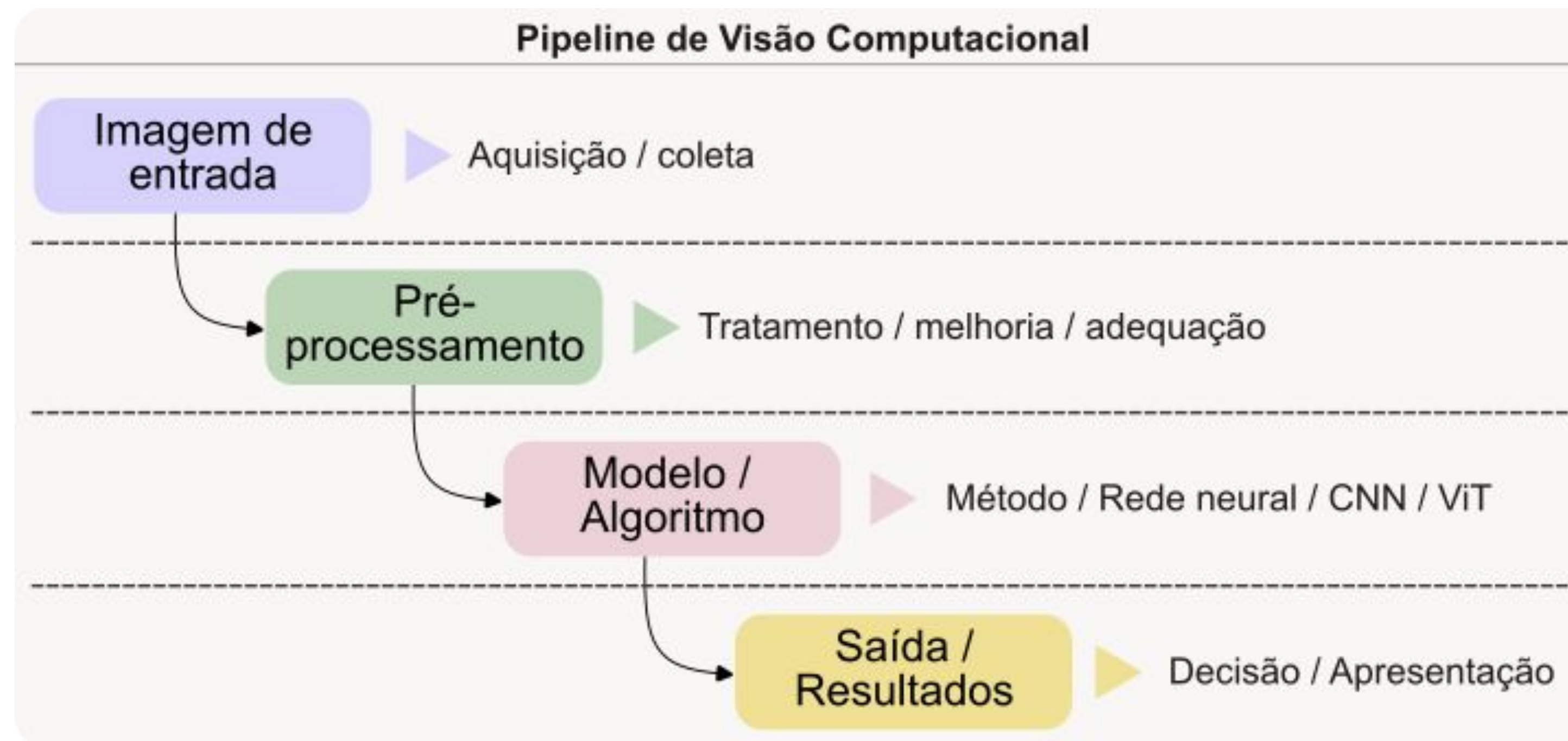
Amplitude

- Visão computacional é um campo muito amplo
- A produção de imagens e vídeos sofreu um aumento exponencial nos últimos 25 anos
- Necessidade de organizar, buscar, ordenar, comparar ou extrair semântica de imagens e vídeos
- Pode impactar diversas áreas da sociedade (inclusive segurança pública)
- Está no centro da revolução de IA
- Implicações éticas (privacidade, estado de vigilância, representatividade, viéses).

Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?

Pipeline clássica

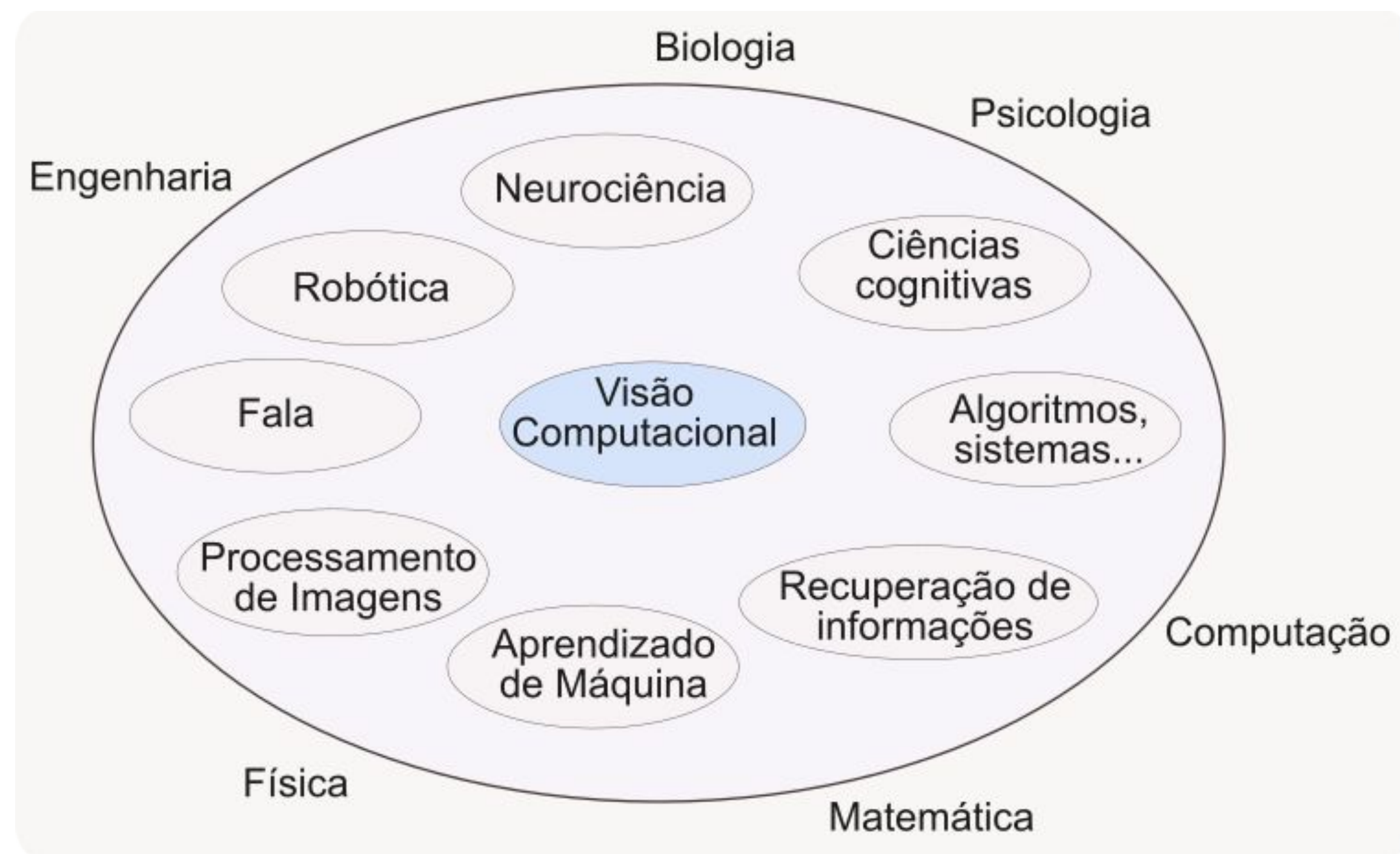


Entender as etapas iniciais do pipeline, incluindo a representação e pré-processamento de imagens é essencial para o desenvolvimento de aplicações de Deep Learning na área de VC.

Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?

Relacionamento com outras áreas



Visão Computacional e IA Generativa

1.1. O que é Visão Computacional?

Desafio: *gap semântico*

Imagem



Representação

23	154	12	214	199	187	232	236
103	206	87	126	211	120	133	255
123	111	45	120	160	127	164	174
243	123	78	144	155	166	125	122
233	212	89	167	16	83	98	101
165	108	123	155	5	19	59	34
199	224	133	233	67	0	7	122
200	211	166	241	33	28	65	211

<< gap semântico >>

Visão Computacional e IA Generativa

1 - Fundamentos de Visão Computacional

1.1. O que é Visão Computacional?

1.2. Representação digital de imagens (PIL e OpenCV)

1.3. NumPy para manipulação de imagens

1.4. Histograma, contraste e realce

1.5. Transformações geométricas e filtros

1.6. Operadores morfológicos

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Pixels

Uma imagem digital é uma **grade retangular de pixels** (picture elements). Cada pixel é a menor unidade de informação de uma imagem e armazena um valor de intensidade ou cor.

Cada imagem tem uma largura (width), altura (height) e canais (cores).

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Após processo de captura, as imagens são armazenadas em representações digitais.

De forma simplificada, as imagens são armazenadas como pixels.

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Pixels

Uma **imagem preto e branca** (grayscale) é representada como uma matriz.

As dimensões dela serão apenas (largura, altura) - possuem apenas um "canal".

Cada pixel é um valor de 0 a 255:

- 0: preto ➤ 255: branco
- Valores intermediários são faixas entre preto e branco (cinza).

23	154	12	214	199	187	232	236
103	206	87	126	211	120	133	255
123	111	45	120	160	127	164	174
243	123	78	144	155	166	125	122
233	212	89	167	16	83	98	101
165	108	123	155	5	19	59	34
199	224	133	233	67	0	7	122
200	211	166	241	33	28	65	211

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Pixels

Uma **imagem colorida**, na representação **RGB**, tem as seguintes dimensões: (largura, altura, 3).

Cada pixel é composto por 3 valores (r, g, b).

Uma imagem colorida, na representação **RGBA** (algumas imagens png), tem as seguintes dimensões: (largura, altura, 4).

Cada pixel é composto por 4 valores (r, g, b, a). O a (ou **alpha**) é um valor de opacidade do pixel. ***Na prática, convertemos imagens RGBA para RGB.***

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Canais e espaços de cor

- **Grayscale (escala de cinza):** 1 canal. Cada pixel = 1 valor de intensidade [0, 255].
shape: (H, W)
- **RGB:** 3 canais — Red, Green, Blue. Cada pixel = 3 valores. shape: (H, W, 3)
- **RGBA:** 4 canais — RGB + Alpha (transparência). shape: (H, W, 4)
- **HSV:** Hue (matiz), Saturation (saturação), Value (brilho). Mais intuitivo para segmentar por cor.
- **BGR:** Mesma informação do RGB, mas ordem invertida. **Padrão do OpenCV** — atenção!
- **YUV** O espaço de cor YUV é um modelo que separa a informação de luminosidade (brilho) da informação de cromaticidade (cor).

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Canais e espaços de cor

Curiosidade: o esquema **YUV** foi criado originalmente para permitir que a televisão em cores fosse compatível com os aparelhos antigos em preto e branco.

Veremos um exemplo de uso na equalização de imagens coloridas. No YUV, o Y (Luma) representa o brilho da imagem (a versão em escala de cinza), sendo o componente que o olho humano percebe com mais detalhes. O U (Crominância Azul) define a diferença entre a luz azul e a luminosidade e o V (Crominância Vermelha) define a diferença entre a luz vermelha e a luminosidade.

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Profundidade de bits

A **profundidade de bits** define quantos valores distintos cada canal pode representar:

Bits	Valores por canal	Tipo numpy	Uso
8 bits	0–255	<code>uint8</code>	Imagens cotidianas (fotos, vídeo)
16 bits	0–65535	<code>uint16</code>	Imagens médicas, astronomia
32 bits float	0.0–1.0	<code>float32</code>	Processamento interno em redes neurais

💡 **Atenção:** Modelos de deep learning normalmente esperam tensores `float32` normalizados em $[0, 1]$. A conversão `uint8` → `float32` é uma das primeiras etapas do pré-processamento.

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Exercícios práticos (criar ambiente)

- Criar pasta “praticas”
- Descompactar arquivos (.zip)
- Abrir powershell na pasta “praticas”
- Executar comandos

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Exercícios práticos (criar ambiente / comandos)

- powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
- uv venv venv-gen-ai --python 3.11.9
- .\venv-gen-ai\Scripts\activate
- uv pip install cmake
- uv pip install wheel
- uv pip install torch torchvision scikit-learn scikit-image matplotlib timm
- uv pip install fastapi gradio pydantic ipython jupyter dlib
- uv pip install opencv-python opencv-contrib-python

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Código - bibliotecas

- PIL
- OpenCV
- Numpy
- Matplotlib

```
from PIL import Image
import cv2
import numpy as np
import matplotlib.pyplot as plt
```


Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Código - abrindo imagem e exibindo

- PIL
- OpenCV
- Numpy
- Matplotlib

```
img_pil = Image.open('../images/vc.png')  
img_np = np.array(img_pil)  
fig = plt.figure(figsize=(6, 4))  
imgplot = plt.imshow(img_np)
```

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Coordenadas da imagem

- PIL
- OpenCV
- Numpy
- Matplotlib

```
img_pil = Image.open('../images/vc.jpg')
print(f"{img_pil.size} (largura x altura)")
# (275, 234) (largura x altura)
img_np = np.array(img_pil)
print(f"{img_np.shape} (altura x largura)")
# (234, 275) (altura x largura)
```

Observe que as **dimensões** de **img_pil** e **img_np** são **invertidas**.

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Coordenadas da imagem



Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Coordenadas da imagem



Supondo que a imagem já foi carregada num objeto numpy (`img_np`), como fazer para colocar uma tarja branca no rosto da mulher?

Passo 1: estimar aproximadamente as coordenadas do rosto.

➤ Dimensões em `img_np` são (altura, largura)

Passo 2: atribuir 255 (branco) para os pixels da área.

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Coordenadas da imagem



```
img_np = np.array(img_pil)  
img_np[40:65, 70:95] = 255
```

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Coordenadas da imagem



```
img_np = np.array(img_pil)  
img_np[40:65, 70:95] = 255
```

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Código - alterando valor de pixels da imagem

- PIL
- OpenCV
- Numpy
- Matplotlib

```
img_pil = Image.open('../images/vc.jpg')  
img_np = np.array(img_pil)  
img_np[0:10, 0:6] = 0  
img_np[0:10, 6:12] = 255
```



Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Código - alterando valor de pixels da imagem

- PIL
- OpenCV
- Numpy
- Matplotlib

```
img_pil = Image.open('../images/vc.jpg')  
img_np = np.array(img_pil)  
altura = img_np.shape[0]  
largura = img_np.shape[1]  
img_np[0:10, largura-12:largura-6] = 0  
img_np[0:10, largura-6:largura] = 255
```

↕
Equivalentes

```
img_np[0:10, -12:-6] = 0  
img_np[0:10, -6:] = 255
```



Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Imagens coloridas

- PIL
- OpenCV
- Numpy
- Matplotlib

```
img_pil = Image.open('../images/vc.jpg')
print(f"{img_pil.size} (largura x altura)")
# (275, 234) (largura x altura)
print(f"Image.mode: {img_pil.mode}")    # RGB, L, RGBA...
print(f'Formato    : {img_pil.format}')  # JPEG, PNG...
print(f'Info extra: {img_pil.info}')     # metadados EXIF etc.
# convertemos para numpy
img_np = np.array(img_pil)
print(f"{img_np.shape} (altura, largura, canais)")
# (234, 275, 3) (altura, largura, canais)
```

Observe que as **dimensões** de **img_pil** e **img_np** são **invertidas**.

Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Código - alterando valor de pixels da imagem

- PIL
- OpenCV
- Numpy
- Matplotlib

```
img_pil = Image.open('../images/vc.jpg')  
img_np = np.array(img_pil)  
img_np[0:10, 0:6, :] = np.array([255, 0, 0])  
img_np[0:10, 6:12] = np.array([0, 255, 0])  
img_np[0:10, 12:18] = np.array([0, 0, 255])
```



Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Código - redimensionando imagens

- PIL
- OpenCV
- Numpy
- Matplotlib

```
img_pil = Image.open('../images/revista.png')
print(img_pil.size) # > (613, 403)
display(img_pil)
r=Image.LANCZOS
img_224x224 = img_pil.resize((224, 224), resample=r)
print(img_224x224.size) # > (224, 224)
display(img_224x224)
```



Perdeu
proporções

Visão Computacional e IA Generativa

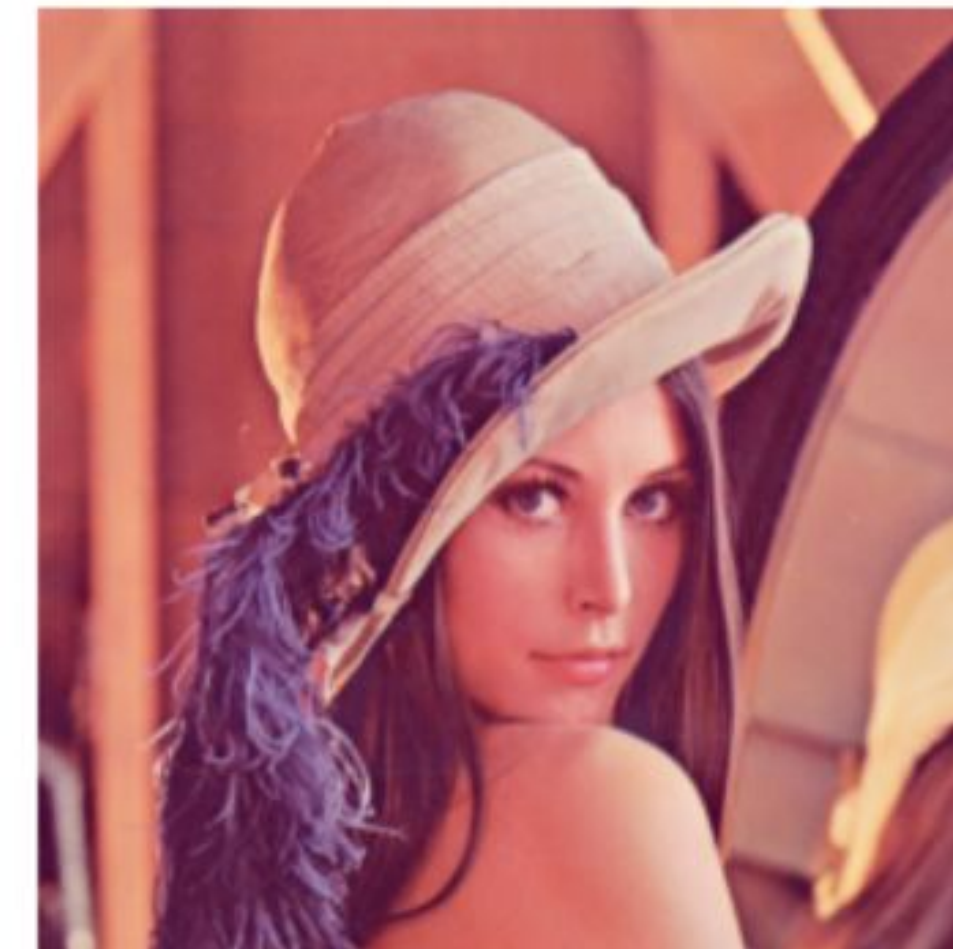
1.2. Representação digital de imagens (PIL e OpenCV)

Abrindo imagens com OpenVC

- PIL
- OpenCV
- Numpy
- Matplotlib

```
img_bgr = cv2.imread('../images/lenna.png') # BGR por padrão
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
fig = plt.figure(figsize=(3, 3))
plt.title("Imagem convertida para RGB")
plt.axis('off')
fig = plt.imshow(img_rgb)
```

Imagem convertida para RGB



Visão Computacional e IA Generativa

1.2. Representação digital de imagens (PIL e OpenCV)

Prática

Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

Observações importantes

- Homogeneidade: Diferente das listas do Python, um array NumPy deve conter apenas um tipo de dado. Se você misturar int e string, o NumPy converterá tudo para string (o tipo mais genérico).
- Performance: Arrays NumPy são fixos em tamanho no momento da criação. Para "adicionar" elementos, o NumPy na verdade cria um novo array, o que pode ser lento se feito repetidamente.
- A Tupla de Formato: Sempre que uma função pedir o formato (como em `np.zeros`), lembre-se de passar os valores entre parênteses extras: (linhas, colunas)

Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

Observações importantes

- Homogeneidade: Diferente das listas do Python, um array NumPy deve conter apenas um tipo de dado. Se você misturar int e string, o NumPy converterá tudo para string (o tipo mais genérico).
- Performance: Arrays NumPy são fixos em tamanho no momento da criação. Para "adicionar" elementos, o NumPy na verdade cria um novo array, o que pode ser lento se feito repetidamente.
- A Tupla de Formato: Sempre que uma função pedir o formato (como em `np.zeros`), lembre-se de passar os valores entre parênteses extras: (linhas, colunas)

Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

```
# Criando vetor a partir de uma lista de inteiros
vetor = np.array([10, 20, 30, 40])
# exibindo algumas propriedades
print(vetor)
print(f"Tipo do objeto: {type(vetor)}")
print(f"Formato (shape): {vetor.shape}")
print(f"Tipo interno (dtype): {vetor.dtype}")
```

```
[10 20 30 40]
Tipo do objeto: <class 'numpy.ndarray'>
Formato (shape): (4,)
Tipo interno (dtype): int64)
```


Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

```
# Criando matriz a partir de uma lista de inteiros
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matriz)
print(f"Dimensão: {matriz.shape}")
print(f"Número de dimensões: {matriz.ndim}")
print(f"Total de elementos: {matriz.size}")
```

```
[[1 2 3]
 [4 5 6]]
Dimensão: (2, 3)
Número de dimensões: 2
Total de elementos: 6
```

Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

```
# especificando o tipo de dado
array_float = np.array([
    [1, 2, 3],
    [4, 5, 6]
], dtype='float32')

print(array_float)
print(f"Tipo dos dados: {array_float.dtype}")
```

```
[[1. 2. 3.]
 [4. 5. 6.]]
Tipo dos dados: float32
```

Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

```
# criando placeholders (funcoes uteis)
zeros = np.zeros((3, 3)) # Requer uma tupla com as dimensões
# Array de um intervalo (similar ao range() do Python)
sequencia = np.arange(0, 10, 2) # (início, fim exclusivo, passo)
# Array com valores linearmente espaçados
espacado = np.linspace(0, 1, 5) # (início, fim, quantidade de números)
print(zeros)
print(sequencia)
print(espacado)
```

```
zeros
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

```
sequencia
[0 2 4 6 8]
```

```
espacado
[0.  0.25 0.5  0.75 1.  ]
```


Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

Broadcasting (propagação)

Algumas operações podem ser realizadas entre objetos de dimensões diferentes.

Nesses casos, o numpy automaticamente tenta fazer broadcasting da operação.

Para que o broadcasting aconteça, o numpy compara os shapes dos dois arrays da direita para a esquerda. Duas dimensões são compatíveis quando:

- Uma delas é 1.
- Elas são iguais.

Exemplo:

```
# somando um escalar em todos elementos do vetor (ou matriz)
vetor = np.array([1, 2, 3])
resultado_escalar = vetor + 10
```


Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

Broadcasting (propagação)

Algumas operações podem ser realizadas entre objetos de dimensões diferentes.

Nesses casos, o numpy automaticamente tenta fazer broadcasting da operação.

Para que o broadcasting aconteça, o numpy compara os shapes dos dois arrays da direita para a esquerda. Duas dimensões são compatíveis quando:

- Uma delas é 1.
- Elas são iguais.

Exemplo:

```
# Matriz + vetor > soma vetor a cada linha da matriz
matriz = np.array([[1, 2, 3], [4, 5, 6]])
print('\nshape da matriz:', matriz.shape) # (2, 3)
vetor_linha = np.array([10, 20, 30])
print('shape do vetor_linha:', vetor_linha.shape) # (3,)
resultado_matriz = matriz + vetor_linha
```

```
[[1 2 3]  [10 20 30]  [[11 22 33]
 [4 5 6]  [14 25 36]]
```


Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

```
# produto escalar / multiplicação de matrizes (np.dot ou operador @)
```

```
# produto escalar = |v1|.|v2|.cos(θ)
```

```
v1 = np.array([3, 2])
```

```
v2 = np.array([4, 0])
```

```
prod_vet = v1 @ v2 # ou v1.dot(v2)
```

```
print(f"Produto escalar {v1}@{v2} = {prod_vet}\n")
```

Produto escalar [3 2]@[4 0] = 12

```
# Matriz (2, 3)
```

```
ma1 = np.array([[1, 2, 3], [4, 5, 6]])
```

```
# Matriz (3, 2)
```

```
ma2 = np.array([[7, 8], [9, 10], [11, 12]])
```

```
# Multiplicação de matrizes
```

```
resultado_dot = np.dot(ma1, ma2)
```

```
# Forma moderna e preferida
```

```
resultado_at = ma1 @ ma2
```

```
print("Resultado da Multiplicação (2,2):\n", resultado_at)
```

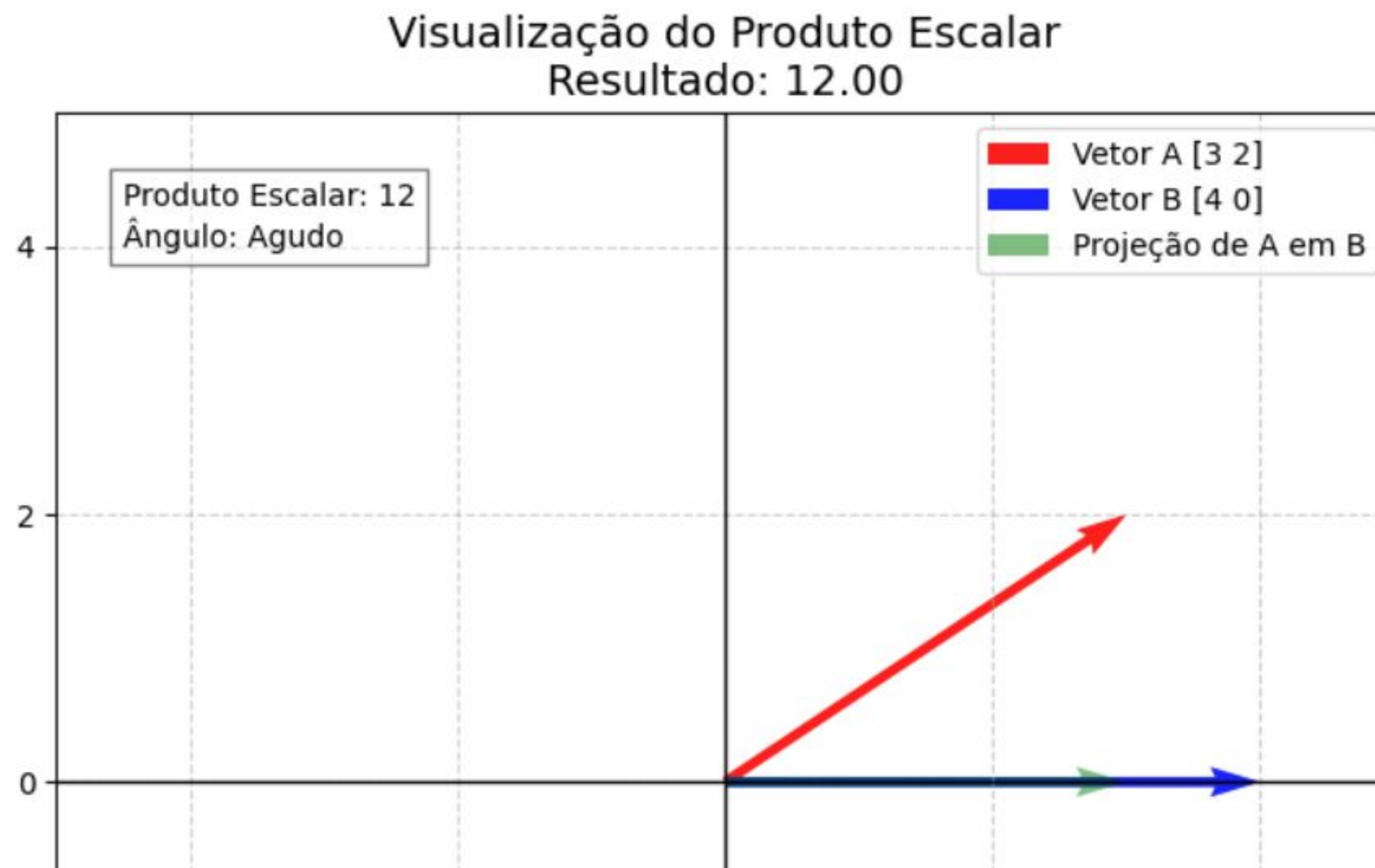
Resultado da Multiplicação (2,2):

[[58 64]

[139 154]]

Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens



Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

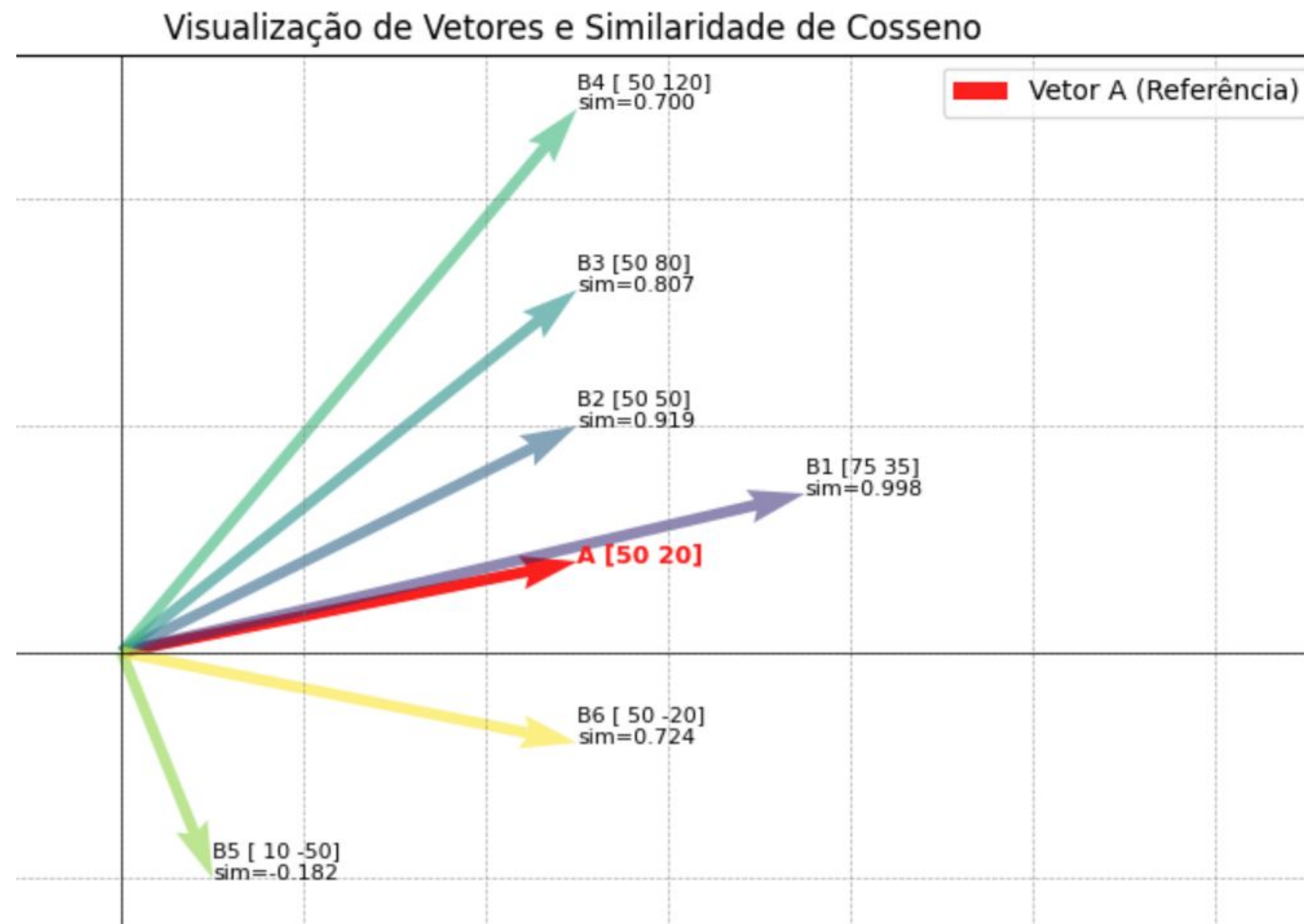
```
# calculo de similaridade de cosseno
# muito útil para comparar a similaridade de dois vetores
# formula:  $\cos(\theta) = (a @ b) / (|a| * |b|)$ 
def sim_cosseno(A, B):
    dot_product = np.dot(A, B)
    norm_A = np.linalg.norm(A)
    norm_B = np.linalg.norm(B)
    if norm_A == 0 or norm_B == 0:
        return 0
    return dot_product / (norm_A * norm_B)
```

```
print('(10, 7) * (10, 7):', sim_cosseno( np.array([10, 7]), np.array([10, 7])) )
print('(10, 7) * (1, 0.7):', sim_cosseno( np.array([10, 7]), np.array([1, 0.7])) )
print('(10, 7) * (10, 10):', sim_cosseno( np.array([10, 7]), np.array([10, 10])) )
print('(10, 7) * (10, 100):', sim_cosseno( np.array([10, 7]), np.array([10, 100])) )
print('ortogonal')
print('(10, 7) * (-7, 10):', sim_cosseno( np.array([10, 7]), np.array([-7, 10])) )
```

```
(10, 7) * (10, 7): 1.0000000000000002
(10, 7) * (1, 0.7): 1.0
(10, 7) * (10, 10): 0.9847835588179368
(10, 7) * (10, 100): 0.6521329826588793
ortogonal
(10, 7) * (-7, 10): 0.0
```


Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens



Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

```
# Redimensionando vetores / matrizes
# função reshape
dados = np.arange(12)
print('shape original:', dados.shape)

# Remodelar para 3 linhas e 4 colunas
matriz_3x4 = dados.reshape(3, 4)
print('shape matriz_3x4:', matriz_3x4.shape)

# Dica : use -1 para o NumPy calcular uma das dimensões automaticamente
matriz_auto = dados.reshape(2, -1) # Ele entende que o -1 deve ser 6
print('shape matriz_auto:', matriz_auto.shape)

print("\nOriginal:", dados)
print("\nReshaped (3,4):\n", matriz_3x4)
print("\nAutomatico (2, -1):\n", matriz_auto)
```

```
shape original: (12,)
shape matriz_3x4: (3, 4)
shape matriz_auto: (2, 6)
```

```
Original: [ 0  1  2  3  4  5  6  7  8  9 10 11]
```

```
Reshaped (3,4):
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
Automatico (2, -1):
```

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```


Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

```
# Argmax (obtendo índice do maior valor)
# voce treinou uma rede neural para analisar imagens e retornar
# as probabilidades das seguintes classes
classes = ['pessoa', 'carro', 'arma', 'moto']
# imagine que probs tem o resultado da inferência de uma imagem
probs = np.array([0.05, 0.12, 0.75, 0.08])
classe_predita = np.argmax(probs)
print(f"A maior probabilidade está no índice: {classe_predita}")
print(f"Logo, o objeto encontrado é da classe {classes[classe_predita]}")
# agora imagine q vc fez uma predicao em lote (batch) de 8 imagens
# e o resultado está em batch_probs
batch_probs = np.array([
    [0.15, 0.25, 0.20, 0.40],
    [0.07, 0.53, 0.25, 0.15],
    [0.02, 0.15, 0.70, 0.13],
    [0.15, 0.10, 0.60, 0.15],
    [0.99, 0.01, 0.00, 0.00],
    [0.33, 0.43, 0.20, 0.04],
    [0.11, 0.00, 0.00, 0.89],
    [0.45, 0.20, 0.01, 0.34],
])
idx_predicoes = np.argmax(batch_probs, axis=1)
print(f"Índices das classes preditas no batch: {idx_predicoes}")
labels_preds = [classes[idx] for idx in idx_predicoes]
print(f"Labels preditos: {labels_preds}")
```

A maior probabilidade está no índice: 2
Logo, o objeto encontrado é da classe arma

Índices das classes preditas no batch:
[3 1 2 2 0 1 3 0]

Labels preditos: ['moto', 'carro', 'arma',
'arma', 'pessoa', 'carro', 'moto', 'pessoa']

Visão Computacional e IA Generativa

1.3. NumPy para manipulação de imagens

Prática

Visão Computacional e IA Generativa

1.4. Histograma, contraste e realce

Prática

Visão Computacional e IA Generativa

1.5. Transformações geométricas e filtros

Prática

Visão Computacional e IA Generativa

1.6. Operadores morfológicos

Prática



IBMEC.BR

 /IBMEC

 IBMEC

 @IBMEC_OFICIAL

 @IBMEC

 **ibmec**